

Relatório de Projeto: Inventário Distribuído (MC714)

Giovanni Mesquita Micaroni

RA: 231702

Repositório GitHub

14 de junho de 2026

Sumário

1	Descrição do Problema	3
2	Algoritmos Escolhidos	3
2.1	Sincronização e Ordem: Relógios Lógicos de Lamport	3
2.2	Exclusão Mútua Distribuída: Ricart-Agrawala	4
2.3	Eleição de Líder: Algoritmo de Bully	4
3	Detalhes de Implementação	4
3.1	Linguagem e Bibliotecas	4
3.2	Sistema de Comunicação	5
3.3	Ambiente de Execução	5
3.4	Testes Realizados	6
4	Resultados de Experimentos	6
4.1	Estrutura dos Logs	7
4.2	Ordenação por Relógios de Lamport	7
4.3	Exclusão Mútua	7
4.4	Eleição de Líder e Tolerância a Falhas	7
5	Comentários sobre a Experiência	8

1 Descrição do Problema

O objetivo deste projeto é implementar um sistema de gerenciamento de inventário distribuído, no qual quatro nós independentes compartilham o controle de um único contador de estoque. O cenário reproduz uma situação recorrente em sistemas distribuídos reais, onde diversos processos precisam operar sobre um mesmo recurso compartilhado, porém sem dispor de memória compartilhada nem de uma autoridade central que coordene os acessos.

Cada nó tenta periodicamente realizar um *checkout*, isto é, retirar uma unidade do estoque, decrementando o contador. Como múltiplos nós podem tentar essa operação ao mesmo tempo, surge a necessidade de garantir que apenas um deles modifique o estado por vez, evitando condições de corrida que levariam a contagens inconsistentes. Além disso, o sistema designa um dos nós como líder, responsável por operações privilegiadas, neste caso, o reabastecimento periódico (*restock*) do inventário. Após cada alteração de estado, o nó que a realizou propaga o novo valor do contador aos demais.

As principais restrições e desafios do problema são:

- **Ordenação de eventos:** como os nós possuem relógios físicos independentes e não sincronizados, é necessário estabelecer uma ordem consistente entre os eventos que ocorrem ao longo do sistema.
- **Exclusão mútua:** o acesso à seção crítica, ou seja, a modificação do contador de estoque, deve ser serializado entre os nós, de modo que nunca dois nós a executem simultaneamente.
- **Coordenação e tolerância a falhas:** o sistema deve eleger dinamicamente um líder e ser capaz de detectar sua eventual falha, promovendo uma nova eleição sempre que necessário.
- **Consistência de estado:** após cada modificação, o novo valor do contador precisa ser difundido aos outros nós, para que todos mantenham uma visão coerente do inventário.

2 Algoritmos Escolhidos

Para atender aos desafios descritos, foram selecionados três algoritmos clássicos da literatura de sistemas distribuídos, cada um responsável por uma dimensão distinta da coordenação.

2.1 Sincronização e Ordem: Relógios Lógicos de Lamport

Como os nós não compartilham um relógio físico comum, utilizou-se o algoritmo de relógios lógicos de Lamport para estabelecer uma ordenação consistente dos eventos. Cada nó mantém um contador lógico que é incrementado a cada evento local. Toda mensagem trocada entre nós carrega o *timestamp* do remetente e, ao receber uma mensagem, o nó destinatário avança seu próprio relógio para o valor $\max(\text{local}, t) + 1$, em que t é o *timestamp* recebido. Dessa forma, garante-se a relação de *happened-before*, em que se um evento a precede um evento b , então o *timestamp* de a é estritamente menor que o de b . Essa ordenação é fundamental tanto para a depuração do sistema quanto para a priorização de requisições no protocolo de exclusão mútua.

Para a implementação desse algoritmo, foi utilizado como referência o código disponível no repositório [1].

2.2 Exclusão Mútua Distribuída: Ricart-Agrawala

A coordenação do acesso à seção crítica é realizada pelo algoritmo de Ricart-Agrawala, um protocolo baseado em troca de mensagens que dispensa um coordenador central. Quando um nó deseja entrar na seção crítica, ele difunde uma requisição (**RequestCS**) a todos os demais nós e só prossegue após receber a confirmação (**CSReply**) de cada um deles.

A decisão de cada nó ao receber uma requisição segue uma regra de prioridade baseada no *timestamp* de Lamport. Caso o nó esteja atualmente na seção crítica, ou possua uma requisição pendente com *timestamp* menor (e portanto de maior prioridade), ele adia o envio da resposta. Caso contrário, responde imediatamente. As respostas adiadas são liberadas no momento em que o nó libera a seção crítica (**release_cs**). Esse mecanismo é fundamental, visto que garante a propriedade de exclusão mútua sem deadlock.

Para a implementação do algoritmo, foi utilizado como referência o código disponível em [2]. Esse código é uma implementação em Java do algoritmo de Ricart Agrawala, e precisou, além de ser convertido para Python, de algumas adaptações para o contexto do projeto, principalmente a centralização das funcionalidades em uma única classe.

2.3 Eleição de Líder: Algoritmo de Bully

A escolha do nó responsável pelas operações privilegiadas é feita pelo algoritmo de *Bully*. Quando um nó inicia uma eleição, ele envia mensagens de **Election** a todos os peers com identificador (ID) superior ao seu. Se nenhum deles responde dentro de um intervalo de *timeout*, o nó se declara líder e difunde uma mensagem de **Coordinator** aos demais. Caso um nó com ID inferior desafie outro de ID superior, este último reinicia o processo de eleição, assumindo a disputa.

A detecção de falha do líder é feita por um mecanismo de *heartbeat*: cada nó envia periodicamente uma mensagem de **Ping** ao líder atual. Se o líder se mostra inacessível, uma nova eleição é disparada automaticamente, garantindo que o sistema sempre convirja para um único líder ativo.

Para a implementação desse algoritmo, não foi encontrada uma referência similar ao que precisava ser contruído. Por este motivo, a implementação precisou ser feita do zero, a partir dos conceitos vistos em sala.

3 Detalhes de Implementação

Nesta seção, são documentados os aspectos práticos da construção do software.

3.1 Linguagem e Bibliotecas

O sistema foi desenvolvido na linguagem *Python*, aproveitando seu ecossistema de bibliotecas. A concorrência interna de cada nó é gerenciada com o módulo padrão **threading**, e o servidor de comunicação e os laços periódicos de execução (*checkout*, *restock*, *heartbeat* e *election*) rodam em *threads* independentes. As estruturas de estado

compartilhado entre essas *threads*, o relógio lógico e o contador de inventário, são protegidas por travas (*locks*), garantindo segurança em ambiente multithread. O desenvolvimento do sistema teve como principal referência o repositório [3], que implementa um exemplo de sistema de comunicação usando Python, gRPC e Docker.

3.2 Sistema de Comunicação

Toda a comunicação entre nós é feita por meio de chamadas de procedimento remoto utilizando o framework *gRPC* [4][5], que adota um modelo síncrono e transiente. A interface de comunicação é definida no arquivo `proto/inventory.proto`, que descreve o serviço `NodeService` e seus respectivos RPCs:

- `RequestCS / CSReply` - requisição e confirmação para o protocolo de exclusão mútua;
- `Election / Coordinator` - mensagens do algoritmo de eleição de líder;
- `Ping` - verificação de disponibilidade do líder (*heartbeat*);
- `SyncInventory` - difusão do novo valor do contador após uma alteração de estado;
- `SendMessage` - *handshake* de inicialização entre os nós.

A partir desse arquivo, os *stubs* Python são gerados automaticamente com a função do *gRPC tools* `grpc_tools.protoc` e armazenados no diretório `generated/`.

A organização do código segue uma separação clara de responsabilidades para cada componente:

- `src/lamport_clock.py` - implementa o relógio lógico thread-safe;
- `src/mutex.py` - implementa a exclusão mútua de Ricart-Agrawala;
- `src/election.py` - implementa a eleição de líder pelo algoritmo Bully;
- `src/inventory.py` - mantém o contador de estoque thread-safe;
- `src/node.py` - ponto de entrada que conecta todos os componentes, inicia o servidor gRPC, aguarda os peers e lança as *threads* periódicas.

3.3 Ambiente de Execução

O sistema foi projetado para ser executado localmente em contêineres, utilizando *Docker* e *Docker Compose* [6]. O comando `docker compose up --build` instancia os quatro nós (`node1-node4`) sobre uma rede Docker compartilhada, na qual cada nó alcança os demais por seus nomes de serviço. Os *logs* de todos os nós são impressos na saída padrão, prefixados pelo ID do nó e pelo *timestamp* de Lamport corrente, o que facilita o acompanhamento da ordem dos eventos durante a execução. A finalização do ambiente é feita com `docker compose down`. Para liberar o espaço ocupado por contêineres inativos, o comando `docker container prune`.

Essa abordagem baseada em contêineres tem a vantagem de reproduzir, em uma única máquina de desenvolvimento, um ambiente de rede com múltiplos processos isolados que se comunicam exclusivamente por mensagens, aproximando-se do comportamento de um sistema verdadeiramente distribuído.

3.4 Testes Realizados

A validação do sistema concentrou-se em verificar a correção dos três algoritmos sob concorrência e falhas parciais. Foram conduzidos os seguintes cenários de teste:

- **Concorrência na seção crítica:** com os quatro nós tentando realizar *checkout* simultaneamente, observou-se nos *logs* que as entradas na seção crítica ocorreram de forma serializada, sem sobreposição, e que o contador de inventário decrementou de maneira consistente.
- **Falha do líder:** simulou-se a queda do nó líder (interrompendo seu contêiner com o comando `docker container stop <container ID>`) para verificar se o mecanismo de *heartbeat* detectava a indisponibilidade e disparava uma nova eleição, levando o sistema a convergir para um novo líder.
- **Consistência de estado:** verificou-se que, após cada modificação do contador, o valor difundido via `SyncInventory` era refletido corretamente em todos os nós.

O trecho a seguir ilustra a implementação do relógio lógico de Lamport, base da ordenação de eventos do sistema.

```
1 class LamportClock:
2     def __init__(self) -> None:
3         self.__time = 0
4         self.__lock = threading.Lock()
5
6
7     def tick(self) -> int:
8         with self.__lock:
9             self.__time += 1
10            return self.__time
11
12     def update(self, t) -> int:
13         with self.__lock:
14             self.__time = max(t, self.__time) + 1
15             return self.__time
16
17     def value(self) -> int:
18         with self.__lock:
19             return self.__time
```

Listing 1: Implementação thread-safe do relógio lógico de Lamport.

Como a implementação do restante dos algoritmos se provou extensa, optou-se por não incluí-las no relatório, mas estas estão disponíveis no repositório do GitHub.

4 Resultados de Experimentos

Diferentemente de aplicações voltadas ao processamento paralelo intensivo, o foco deste projeto está na *correção* da coordenação distribuída, e não no ganho de desempenho bruto. Por essa razão, os experimentos foram conduzidos com o objetivo de validar o comportamento esperado dos protocolos, e a principal ferramenta de validação foram os *logs* de execução. Cada nó imprime suas ações na saída padrão, prefixadas pelo identificador do nó e pelo *timestamp* de Lamport corrente, o que permite reconstruir a ordem dos eventos e auditar o comportamento do sistema durante a concorrência e as falhas.

4.1 Estrutura dos Logs

Todas as linhas de *log* seguem o formato `[node<ID> ts=<timestamp>]`, em que ID identifica o nó e `ts` é o valor atual de seu relógio de Lamport. Esse prefixo é fundamental para a análise, pois permite ordenar logicamente eventos originados em nós distintos, mesmo na ausência de um relógio físico comum. O exemplo a seguir ilustra um trecho representativo da saída agregada dos quatro nós.

```
1 node2-1 | [node2 ts=21] REQUESTING CS
2 node3-1 | [node3 ts=22] REQUESTING CS
3 node2-1 | [node2 ts=25] ENTERING CS      inventory=20
4 node2-1 | [node2 ts=26] CHECKOUT OK      inventory now 19
5 node2-1 | [node2 ts=27] EXITING CS
6 node1-1 | [node1 ts=28] synced inventory=19 from node2
7 node3-1 | [node3 ts=28] synced inventory=19 from node2
8 node4-1 | [node4 ts=29] synced inventory=19 from node2
9 node3-1 | [node3 ts=30] ENTERING CS #esperou o release de node2
10 node3-1 | [node3 ts=31] CHECKOUT OK      inventory now 18
11 node3-1 | [node3 ts=32] EXITING CS
12 node1-1 | [node1 ts=33] synced
```

Listing 2: Trecho representativo dos logs de execução.

4.2 Ordenação por Relógios de Lamport

A inspeção dos *timestamps* confirmou a propriedade de ordenação lógica: ao longo de toda a cadeia causal de eventos, os valores de Lamport apresentaram-se monotonicamente crescentes. Sempre que uma mensagem era recebida, o relógio do destinatário avançava para um valor estritamente maior que o do remetente, evidenciando a relação de *happened-before* entre o envio e o recebimento. Isso valida que a sincronização lógica está corretamente implementada e que os logs refletem fielmente a ordem causal das operações.

4.3 Exclusão Mútua

Em cenários nos quais os quatro nós tentavam realizar *checkout* simultaneamente, a leitura dos *logs* mostrou que as entradas na seção crítica ocorreram de forma estritamente serializada. Não houve nenhum registro de dois nós relatando estar na seção crítica ao mesmo tempo, e cada bloco de execução exibia o padrão esperado: difusão do **RequestCS**, coleta de todas as respostas, entrada na seção crítica, atualização do estoque e liberação das respostas adiadas. As mensagens de **SyncInventory** subsequentes confirmavam que o novo valor do contador era propagado e refletido de maneira consistente em todos os nós.

4.4 Eleição de Líder e Tolerância a Falhas

Para validar o algoritmo Bully, simulou-se a falha do líder interrompendo seu contêiner durante a execução. Os *logs* permitiram acompanhar toda a sequência de recuperação: o mecanismo de *heartbeat* de outro nó registrava a indisponibilidade do líder, uma nova eleição era disparada com o envio de mensagens **Election** aos peers de ID superior e, na ausência de resposta destes, o nó vencedor anunciava-se difundindo a mensagem **Coordinator**. Tanto na inicialização quanto após a falha simulada, o sistema convergiu corretamente para um único líder ativo, retomando em seguida as operações de *restock*.

5 Comentários sobre a Experiência

O desenvolvimento deste projeto proporcionou a oportunidade de colocar em prática conceitos que, em sala de aula, costumam permanecer apenas como teoria. Implementar os algoritmos de Lamport, Ricart-Agrawala e Bully evidenciou que a maior dificuldade de um sistema distribuído raramente está na lógica de cada algoritmo isolado, mas sim na sua coexistência sob concorrência real.

A principal dificuldade encontrada por mim foi lidar com condições de corrida entre as diferentes *threads* de cada nó, que exigiram cuidado na proteção do estado compartilhado por travas. A depuração de *bugs* distribuídos também se mostrou particularmente desafiadora, já que, por não haver uma linha de execução única, problemas dependiam da ordem de chegada das mensagens, o que tornou os *logs* ordenados por *timestamp* de Lamport uma ferramenta fundamental para entender o que de fato estava ocorrendo. Além disso, lidar com novas bibliotecas e um contexto fortemente multithread se provou desafiador, visto que não é um contexto que lido frequentemente.

Por fim, o projeto deixou clara a diferença entre projetar um sistema no papel e executá-lo em uma rede imperfeita, sujeita a atrasos e falhas. Lidar com a detecção de falhas do líder e com a reorganização do sistema após uma queda ressaltou a percepção de que tolerância a falhas não é um detalhe, mas um requisito central no projeto de qualquer sistema distribuído. A experiência contribuiu significativamente para consolidar minha compreensão da área e para o meu entendimento sobre sistemas concorrentes e distribuídos.

Referências

- [1] LAMPORT Clocks. *GitHub com implementação do Relógio de Lamport*. Disponível em: <https://github.com/AnolChakraborty/Lamport-Clock-Simulator/blob/main/main.py>.
- [2] RICART AGRAWALA Mutex *Implementação de referência Ricart Agrawala*. Disponível em: <https://github.com/joaopmgd/RicartAgrawala>.
- [3] Projeto: *Referência Geral para o Projeto*. Disponível em: <https://github.com/flavienbwk/gRPC-Python-Docker-Example>.
- [4] gRPC. *Documentação Oficial*. Disponível em: <https://grpc.io/docs/>.
- [5] gRPC Tools Exemplos. *Exemplos*. Disponível em: https://github.com/grpc/grpc/blob/v1.81.0/examples/protos/route_guide.proto.
- [6] DOCKER. *Documentação Docker Compose*. Disponível em: <https://docs.docker.com/compose/>.